Simple linear regression

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Distinguish regression problems from classification problems, and supervised learning from unsupervised learning
- Explain the linear equation behind linear regression and interpret its slope and intercept
- Build, train, and evaluate a simple linear regression model using scikit-learn
- Apply evaluation metrics (MAE, MSE, RMSE, R^2) to judge how well a model performs

2. Detailed Explanation

Why linear regression still matters

Linear regression is a machine learning algorithm that models the relationship between an input variable and an output variable using a straight line. It is not obsolete — it forms the conceptual basis for more advanced algorithms such as support vector machines, neural networks, and deep neural networks. When the underlying data truly has a linear relationship, linear regression can outperform far more complex algorithms, including neural networks. It is also highly interpretable, fast to train, and commonly used as a baseline model: a simple model used for comparison against other machine learning algorithms.

The word "simple" in "simple linear regression" means the model uses only one input feature at a time. Multiple linear regression uses several input features together and is a separate extension of the same idea.

Classification vs. regression problem statements

Every machine learning problem uses an input variable `X` and an output variable `Y`. The goal is to build a model that uses `X` to produce `Y`. When there is more than one input variable, they are written individually as `X1, X2, X3, … Xn`, and the full set is referred to collectively as capital `X`.

Every real-world machine learning problem falls into one of two categories, based entirely on the nature of `Y`:

- Classification: `Y` is a discrete output variable — it can only take a fixed, countable set of values, with no continuous range between them.
- Regression: `Y` is a continuous output variable — it can take any value within a range, so there is no fixed, countable set of outputs.

Classification examples include sentiment classification, spam classification, and tumor prediction. Sentiment classification converts text such as Amazon reviews into numeric vectors, since algorithms work with numbers rather than raw text, and outputs a label of positive, negative, or neutral. Spam classification separates legitimate email from spam, and tumor prediction separates benign from malignant cases to prioritize urgent treatment.

Regression examples include house price prediction (ranging from ~1 lakh to billions in India), temperature forecasting, solar energy forecasting, and stock price forecasting. Salary prediction also fits here — for instance, TCS salaries range from ~30,000 to 10 lakhs depending on role. Marks or percentage prediction from past performance is another example.

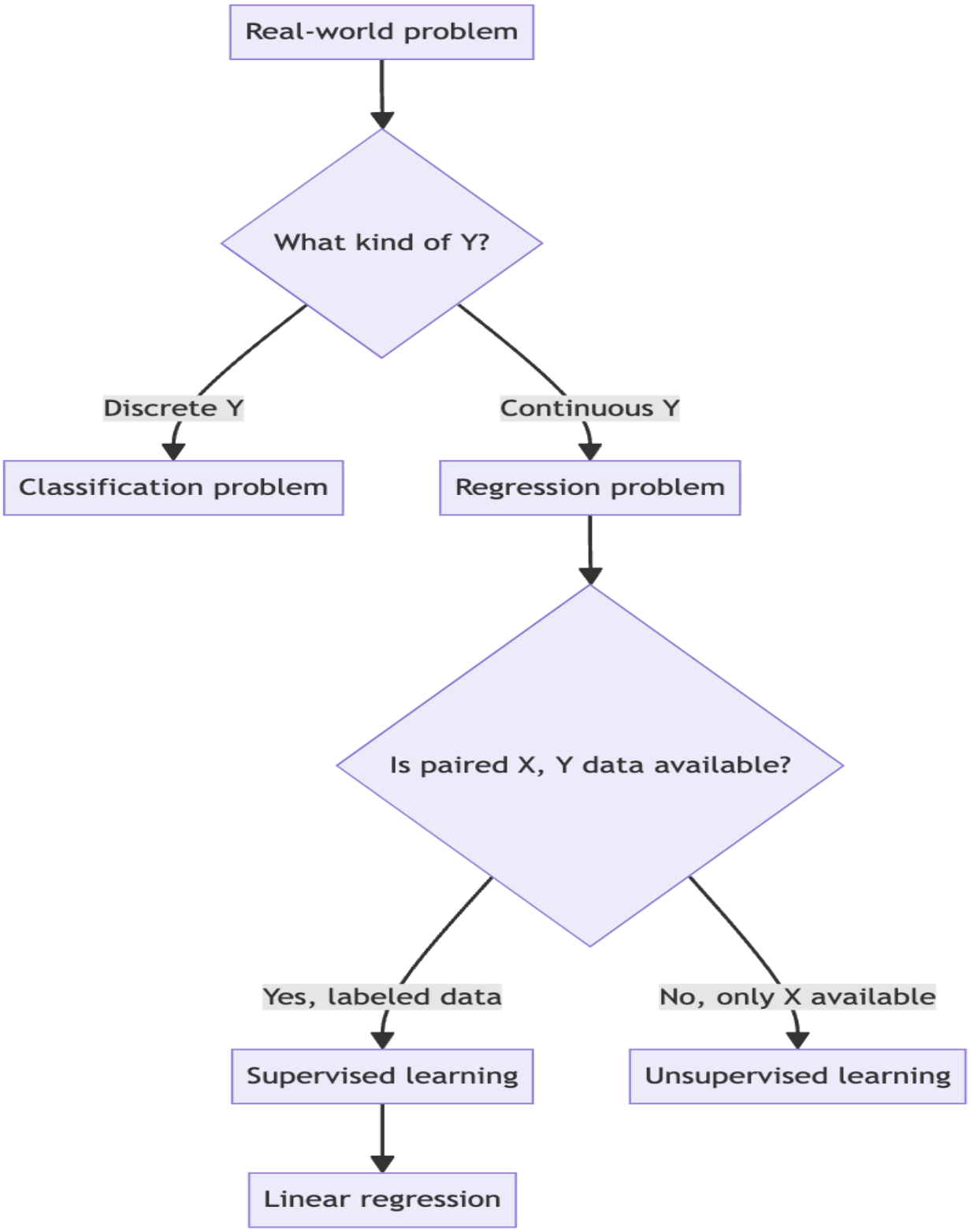
Once a problem is identified as regression — the output is continuous — linear regression is the algorithm used to solve it. To determine which category a problem falls into, inspect the `Y` variable directly, count distinct values with a script, or reason from the problem statement.

Supervised vs. unsupervised learning

Independent of whether a problem is regression or classification, machine learning algorithms also split into two families based on data availability:

> Supervised algorithms: used when the historical data set contains paired `X` and `Y` values, called a labeled data set — for example, employee records (`X`) together with their known salaries (`Y`). The model is trained on this paired data so it can produce an accurate `Y` for a new `X`.
> Unsupervised algorithms: used when only `X` is available and `Y` is missing, called an unlabeled data set — for example, patient records without disclosure of whether each patient actually has a tumor.

Supervised algorithms generally outperform unsupervised algorithms because they have more information to learn from: the training data explicitly shows which `Y` corresponds to which `X`. Linear regression is a supervised algorithm — it is trained on paired `(X, Y)` data such as employee records paired with salaries. Since this lesson focuses on regression, `Y` is always treated as continuous.



What linear regression models

Linear regression models the relationship between an input (independent) variable and an output (dependent) variable. In the simplest case there is a single input variable, `X1`. `Y` is called the dependent variable because it depends on `X1`. For example, house price depends on house size — size is the independent variable, price is the dependent variable.

The algorithm is called "linear" because it assumes a linear relationship: as `X` varies, `Y` varies in a straight-line pattern. In practice this relationship is rarely a perfect straight line — there is often a slight variation or slope deviation — but linear regression models these near-linear relationships well.

Other real-world linear relationship examples:

predicting house price from house size/area
predicting employee salary from years of experience
predicting exam marks from hours of study
blood pressure or sugar levels potentially associated with a person's age or weight

The input variable `X` has several interchangeable names: predictor, feature, and attribute all mean the same thing. Python code commonly uses "attribute". `Y` is referred to as the output variable.

Key properties of linear regression:

Easy to understand and fast to train, because it assumes the `X`-`Y` relationship is linear.
Highly interpretable, because its formula is very simple — this is exactly why it serves as the foundation for understanding more complex algorithms.
Works well as a baseline model for comparison against other machine learning algorithms.
If the data set genuinely has a linear relationship, it can outperform more complex algorithms, including neural networks.

The linear equation

The traditional equation of a straight line is:

```
y = mx + c
```

`m` is the slope — the coefficient multiplied with `x` to obtain `y`. Geometrically, it represents the angle of the line.
`c` is the intercept — the value of `y` when `x` is 0. It represents the starting point of the line.

In machine learning, the same equation is rewritten using weight notation:

```
y = w0 + w1 * x1
```

Here `w0` corresponds to `c` (the intercept) and `w1` corresponds to `m` (the slope for feature `x1`). This can also be written as `w0*x0 + w1*x1`, where `x0` is always fixed at 1.

Think of the intercept (`w0`/`c`) like a new employee with zero years of experience who still needs a starting salary. That base salary — the value of `Y` when `X` is not yet available — is the intercept. As the employee gains experience, their salary increases according to the slope (`w1`).

Watch out for: don't confuse `w0` (the base value when the feature is zero) with `w1` (the rate of change per unit of the feature). Mixing them up will make the equation predict wildly wrong values.

Worked examples and the idea of error

Using a simplified, perfectly linear synthetic house-price data set (prices in dollars, house size in "gaj," an Indian unit of land area):

Size (gaj)	Price ($)
100	1,010
200	2,010
300	3,010

The correct underlying equation is `price = 10 + 10 * size`, i.e., `w0 = 10` (base price with no house at all) and `w1 = 10` (price increase per unit of size). Checking: `10 + 10×100 = 1,010`; `10 + 10×200 = 2,010`; `10 + 10×300 = 3,010` — all consistent.

If incorrect weights are used instead — say `w0 = 5` and `w1 = 5` — the equation becomes `price = 5 + 5 * size`. This predicts `5 + 5×100 = 505` for a 100-gaj house, while the actual price is `1,010`. This gap between the predicted value ($\hat{y}$, "y-hat") and the actual value `y` is called the error.

The goal of linear regression is to find `w0` and `w1` that minimize this error across the training data. With small data sets, humans can infer optimal weights manually. For large data sets with thousands or lakhs of points, finding weights by hand becomes infeasible — this is exactly what the algorithm automates.

A second worked example relates hours of study (`X`) to marks (`Y`). A student can score a base of 20 marks even without studying (`w0 = 20`), and every additional hour of study increases marks by approximately 8 (`w1 = 8`):

```
marks = 20 + 8 * hours_of_study
```

Training data, test data, and the model

Available labeled data is split into two parts:

Training data: shown to the algorithm so it can learn (fit) the optimal weight parameters.
Test data: withheld from training entirely, then used afterward to check how well the trained model generalizes to unseen data.

Once the algorithm produces optimal weights (`w0`, `w1`), the resulting linear equation is the model — conceptually, a function that maps `X` to a predicted `Y`. This trained function can then be applied to genuinely new `X` values, such as predicting salary for a brand-new employee.

Implementing linear regression with scikit-learn

The implementation uses scikit-learn, a Python machine learning library providing both linear and non-linear algorithms. A separate library, Keras, is used later specifically for deep neural network algorithms; scikit-learn covers the more traditional/fundamental algorithms, including linear regression.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

`pandas` builds the data frame holding the data set.
`numpy` handles numeric operations and preprocessing (such as computing a square root later); pandas itself relies on NumPy internally.
`matplotlib.pyplot` (aliased `plt`) is the plotting module used to visualize the data.
`LinearRegression`, from `sklearn.linear_model`, is the class used to implement the algorithm.
`train_test_split`, from `sklearn.model_selection`, splits the full data set into training and test portions.
`mean_absolute_error`, `mean_squared_error`, and `r2_score`, from `sklearn.metrics`, are the evaluation functions.

As an optional aside on terminology: `train_test_split` is called a *function* because it performs a single, direct action (splitting data). `LinearRegression` is a *class* because training a model is multi-step — it groups related operations like `fit` and `predict` calls on an object from that class.

A synthetic data set of 10 employee records is built, with `experience` (years) as the single feature and `salary` as the target:

Experience (years)	Salary
1	30,000
2	35,000

5	58,000

The data is visualized with a scatter-style plot:

```
plt.xlabel("Experience")
plt.ylabel("Salary")
plt.show()
```

Because there is only a single feature, the relationship between `X` (experience) and `Y` (salary) can be visualized directly in two dimensions. With two or more features, direct visualization on a 2D screen is not possible — a topic addressed separately for multiple linear regression. The plotted data showed an almost perfectly straight-line relationship, starting from a base salary around 30,000.

Splitting the data and fitting the model

```
X = df["Experience"]
y = df["Salary"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

`test_size=0.2` reserves 20% of the data points for testing (2 of the 10 rows) and the remaining 80% (8 rows) for training.
The split is random by default — it does not simply take the first 80% and last 20%; it randomly selects which rows go to training versus testing.
`random_state=42` fixes the random seed so the split is reproducible: rerunning the notebook, or having multiple people run the same code, always produces the same train/test split. Omitting `random_state` produces a different split — and therefore different learned weights — on every run.

Fitting the model:

```
model = LinearRegression()
model.fit(X_train, y_train)

print(model.coef_)        # slope (w1)
print(model.intercept_)   # intercept (w0)
```

`LinearRegression()` acts as a constructor, creating a new object (named `model` here). `model.fit(X_train, y_train)` is the training step — the most time-consuming instruction. It produces the optimal weight parameters. The fitted coefficient (slope, `w1`) for experience was approximately 7,629, meaning salary increases by ~$7,629 per year. The intercept (`w0`, base salary with zero experience) was approximately 20,163.

Making predictions

Predicting on the held-out test set uses only `X_test` (not `y_test`, since predicting `Y` is the whole point):

```
pred = model.predict(X_test)
```

For the two held-out test rows: the actual salary for 9 years was 90,000 (predicted ~88,887), and for 2 years was 35,000 (predicted ~35,422). These predictions were very close to actual values, indicating a strong fit consistent with the near-linear data.

The trained model can also predict for a brand-new, unseen data point beyond the test set:

```
model.predict([[20]])  # 20 years of experience
```

This is useful to estimate salary for a brand-new employee based on years of experience. A validated model can be handed to a company (for example, an HR team) to generate salary estimates for future hires using historical data.

Evaluating model performance

Once a model generates predictions on the test set, several standard evaluation metrics — none specific to linear regression, but broadly applicable — assess performance:

Metric	What it measures
Mean Absolute Error (MAE)	Average of the absolute differences between actual and predicted values. The absolute value (mod) prevents negative and positive errors from canceling out.
Mean Squared Error (MSE)	Like MAE, but each difference is squared instead of taking its absolute value. Squaring also converts negative differences to positive, and amplifies larger errors, making them more visible.
Root Mean Squared Error (RMSE)	The square root of MSE — a normalized version of MSE, back on the original scale of the target variable.

R^2 score (R-squared)	Conceptually similar to a correlation coefficient between actual and predicted values, measuring whether they move in the same pattern. Ranges from 0 (no correlation) to 1 (perfect correlation). Higher is better — unlike the error metrics above, which should be minimized.

```
mae = mean_absolute_error(y_test, pred)
mse = mean_squared_error(y_test, pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, pred)
```

Results on the test set (2 held-out points): MSE ≈ 797,000, MAE ≈ 706, RMSE ≈ 893, R^2 ≈ 0.99. An R^2 this close to 1 indicates the model fits the data extremely well. RMSE lands back on the original dollar scale of salary rather than the squared scale. Results on the training set (8 points) were similar — R^2 also approximately 0.99, with an average deviation of roughly $1,137 across the training points.

Watch out for: always report performance on the test set, never the training set. The model never saw test `Y` values during training, so test performance reflects true generalization. Given salary in thousands, an error of a few hundred to ~$1,000 is acceptable. Tolerance depends on context — this judgment belongs to the developer.

Clarifications and good practices

A few points worth keeping straight:

Linear regression vs. logistic regression: despite both having "regression" in the name, they are unrelated in function. Linear regression is a regression algorithm for continuous output variables. Logistic regression, despite its name, is a classification algorithm for discrete output variables.
Is it fine to just use linear regression if it performs well? Yes. If a data set has a linear (or close-to-linear) relationship between `X` and `Y`, linear regression can itself achieve very high performance, with no need for a more complex algorithm. In many real-world scenarios, however, the relationship is not linear, in which case linear regression is not sufficient.
R^2 on train data vs. test data: R^2 measures the same thing in both cases — correlation between actual and predicted `Y`. The numeric values can differ because the underlying data points differ, but the interpretation is identical. Reported performance should always come from the test-set R^2, not the train-set R^2.
How to improve model accuracy: two strategies apply. First, compare against other algorithms (e.g., multi-layer perceptron) to see if performance improves. Second, add relevant features beyond experience — such as skills, education level, or role relevance. More features generally improve accuracy.

3. Key Takeaways

A problem is regression if `Y` is continuous, classification if `Y` is discrete. Linear regression solves regression problems and is supervised — trained on paired `(X, Y)` labeled data.
The linear equation `y = w0 + w1*x1` (equivalent to `y = mx + c`) captures the relationship, where `w0` is the intercept (base value) and `w1` is the slope (rate of change).
The gap between a predicted value ($\hat{y}$) and the actual value (y) is the error; linear regression finds the `w0` and `w1` that minimize this error across the training data.
A scikit-learn workflow is: split data, fit a model, predict on test set, then evaluate with MAE, MSE, RMSE, R^2 — always report test metrics.
R^2 closer to 1 means predictions closely track actual values, while MAE, MSE, and RMSE are error measures that should be minimized rather than maximized.

Mental model: Think of linear regression as drawing the best straight line through your data. The intercept sets where the line starts, and the slope sets how steeply it rises. The error metrics then tell you how closely that line tracks reality.